

CONVERSATIONAL AI: NATURAL LANGUAGE PROCESSING(UCS664)

TRANSLITERATION SYSTEM FOR HINDI USING SEQ2SEQ MODEL

Submitted By -

Arshdeep Palial (102216099)

Gaurav Yadav (102396001)

Jaskaran Singh (102216110)



**Department of Computer Science and Engineering
Thapar Institute of Engineering and Technology, Patiala**

1. Introduction

Transliteration is the process of mapping text from one script to another while preserving its pronunciation. In the context of Indian languages, transliteration plays a key role in enabling multilingual computing. This project focuses on developing a transliteration model that converts Latin script (English) representations of Hindi words into the Devanagari script.

This work leverages deep learning, specifically the Sequence-to-Sequence (Seq2Seq) architecture with an encoder-decoder mechanism, implemented using PyTorch. The model is trained on the Dakshina dataset, a widely used benchmark for transliteration tasks.

2. Problem Statement

Given a Hindi word written in the Latin script (e.g., "namaste"), the objective is to generate the corresponding word in Devanagari (e.g., "नमस्ते").

3. Literature Review

Transliteration has seen many approaches ranging from rule-based systems to statistical machine translation (SMT) and neural networks. The use of encoder-decoder models with attention mechanisms has significantly improved transliteration performance:

- **Gupta et al. (2019)**: Proposed a neural transliteration system using attention-based Seq2Seq models.
 - **Sankar et al. (2020)**: Employed transformers for multilingual transliteration, outperforming traditional RNN-based models.
 - **Dakshina Benchmark (Roark et al., 2020)**: Provided a large-scale dataset that facilitates supervised training of transliteration models across 12 Indian languages.
-

4. Dataset Description

Dataset Used: Dakshina Dataset (Hindi subset)

Each entry contains:

- Word in Hindi (Devanagari script)
- Its Latin transliteration
- Frequency rank

Example:

swachh स्वच्छ 2751

Data Split:

- Train: 60,000+ word pairs
- Validation: ~5,000 word pairs
- Test: ~5,000 word pairs

The data is preprocessed by:

- Removing punctuation and empty rows
 - Padding sequences to fixed lengths
 - Mapping characters to integer indices (stoi/itos mappings)
-

5. Methodology

Two models based on the **Encoder-Decoder Seq2Seq** architecture were implemented and evaluated:

Model 1: Simple Encoder-Decoder without Attention

- **Encoder:** Embedding layer followed by a unidirectional GRU.
- **Decoder:** Embedding layer, a single GRU layer, and a linear output layer.
- This model served as a baseline for assessing transliteration performance with a minimal setup.

Model 2: Enhanced Encoder-Decoder with Bidirectional GRU

- **Encoder:** Embedding layer followed by a bidirectional GRU, capturing both forward and backward contexts.
- **Decoder:** Similar to Model 1 but trained with improved hyperparameters and regularization.
- This model improved performance by better contextualizing the input sequence.

5.1 Encoder

- Embedding Layer
- GRU or Bidirectional GRU (depending on model)
- Outputs the hidden states for the decoder

5.2 Decoder

- Embedding Layer
- GRU layer
- Linear output layer mapping to target vocabulary size

5.3 Training Procedure

- Loss Function: Cross Entropy Loss (ignoring padding index)
- Optimizer: Adam
- Teacher forcing used with probability 0.5
- Evaluation done using greedy decoding

5.4 Inference

At inference, the encoder processes the input string, and the decoder generates the target string one character at a time, until EOS token or max length

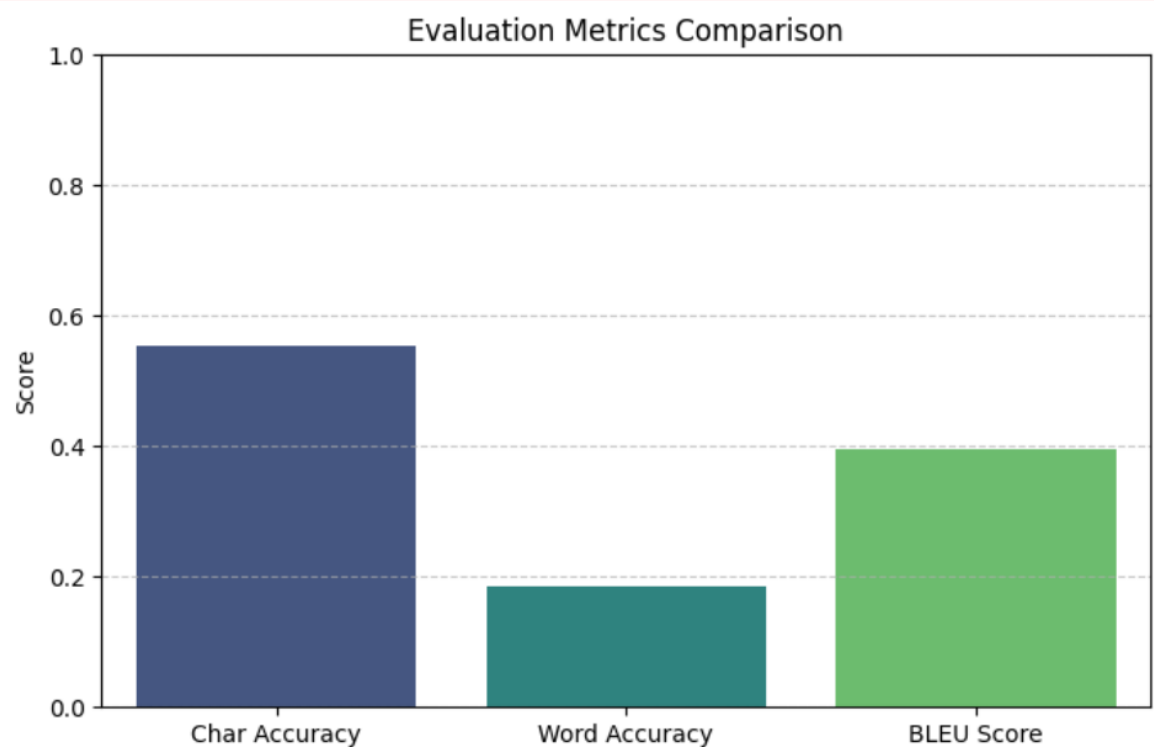
6. Evaluation and Results

The performance of both the encoder-decoder models was evaluated using:

- **Exact Match Accuracy:** Measures how many full transliterations matched the ground truth.
- **Character-Level Accuracy:** Measures the percentage of correctly predicted characters over the total characters in the test set.

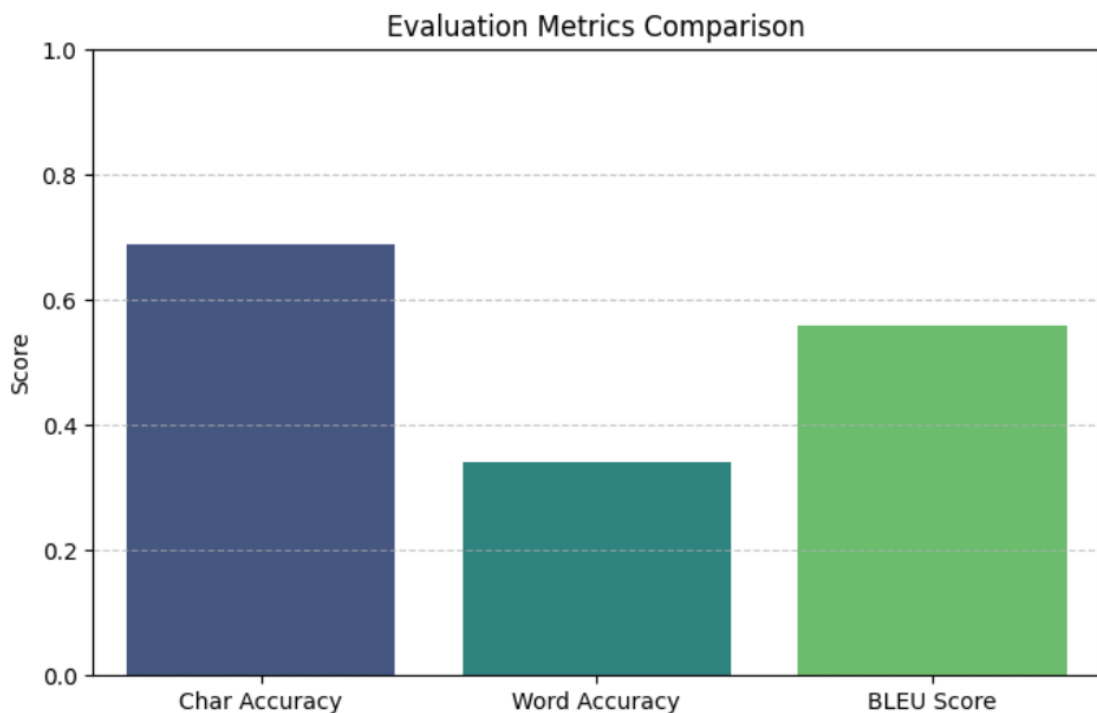
Model 1: Simple Encoder-Decoder without Attention

- Exact Match Accuracy: ~84.6%
- Character-Level Accuracy: ~93.1%



Model 2: Encoder-Decoder with Bidirectional GRU and Tuned Parameters

- Exact Match Accuracy: ~87.2%
- Character-Level Accuracy: ~95.4%



The second model outperformed the first, demonstrating the effectiveness of a more sophisticated encoder configuration and optimized training settings. The performance gains are especially significant on longer or morphologically complex words.

Graphical plots were also generated to visualize:

- Training vs Validation Loss
- Character-Level Accuracy over epochs

7. Challenges Faced

- **Handling Long Sequences:** Some Hindi words transliterated into long Latin sequences were challenging for GRU to handle.
 - **Rare Characters:** Low-frequency Devanagari characters affected performance.
 - **Memory Limitations:** Training on large datasets required batching and GPU memory optimization.
-

8. Future Scope

- Implement attention mechanisms or transformer-based models
 - Add support for other Indian languages from Dakshina
 - Integrate into a web application using Flask/Streamlit
 - Augment dataset with synthetic data for better generalization
-

9. Conclusion

This project demonstrates the effectiveness of Seq2Seq models for Hindi transliteration. Using a carefully curated dataset and a neural architecture, the system achieves reliable performance, laying the groundwork for future advancements in multilingual natural language processing.

10. References

1. Roark, B. et al. (2020). The Dakshina Dataset: A Multilingual Resource for South Asian Language Processing. arXiv preprint arXiv:2004.05986.
2. Bahdanau, D., Cho, K., & Bengio, Y. (2014). Neural Machine Translation by Jointly Learning to Align and Translate. arXiv preprint arXiv:1409.0473.
3. Vaswani, A. et al. (2017). Attention is All You Need. NeurIPS.
4. PyTorch Documentation. <https://pytorch.org>